

## Topic 1 - MLP

**Huber loss** behaves like MAE when input is large but behaves like MSE when small.

**Overfitting:** model learns noise, train acc > val acc, reduce by reducing complexity, regularise, early stopping, data augmentation, smaller lr or batch size.

**Underfitting:** make more complex, train longer, lr too high may prevent convergence, lr too low may slow it excessively, try different optimiser or activations, feature engineer.

## Topic 3 - CNNs

**Key Idea:** learn hierarchical features, low-level (edges) → mid-level (textures) → high-level (objects). Weights are shared.

$I(x + i, y + j)$ : pixel values in local window;  $f(i, j)$ : filter/kernel weights (feature detector);  $I'(x, y)$ : output pixel.

Params per filter =  $n_h \times n_w \times n_{channels} + 1(bias)$

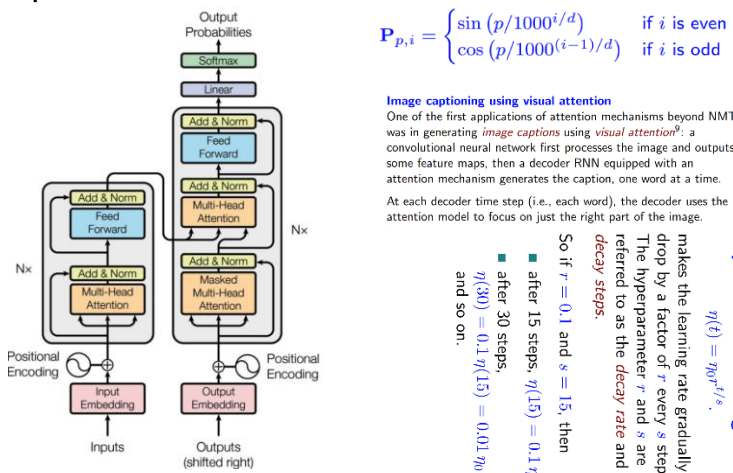
**Stride:** step size between receptive fields.

- Larger stride = smaller output (downsampling).

**Padding:** adds zeros around borders.

- "valid" = no padding → smaller output.
- "same" = keep same spatial size.

## Topic 8 - Transformers

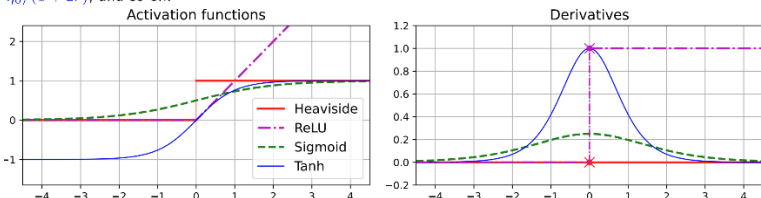


**Power scheduling:** Setting the learning rate as a function of the step number  $t$ , as follows:

$$\eta(t) = \frac{\eta_0}{(1 + rt/s)^c}$$

where  $\eta_0$  is the initial learning rate, the power  $c$  (typically set to 1),  $r$  is the decay rate, and  $s$  is the number of decay steps.

So, after  $s$  steps (i.e., when  $t = s$ ), the learning rate is down to  $\eta_0/(1 + r)$ . After another  $s$  steps (i.e., when  $t = 2s$ ), it is down to  $\eta_0/(1 + 2r)$ , and so on.



LeakyReLU <sub>$\alpha$</sub> ( $z$ ) =  $\max(\alpha z, z)$

$\text{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$   $\sigma(z) = \frac{1}{1 + e^{-z}}$

SELU( $z$ ) =  $s \text{ELU}_{1.6733}(z)$

Swish <sub>$\beta$</sub> ( $z$ ) =  $z\sigma(\beta z)$

softplus( $z$ ) =  $\log(1 + \exp(z))$

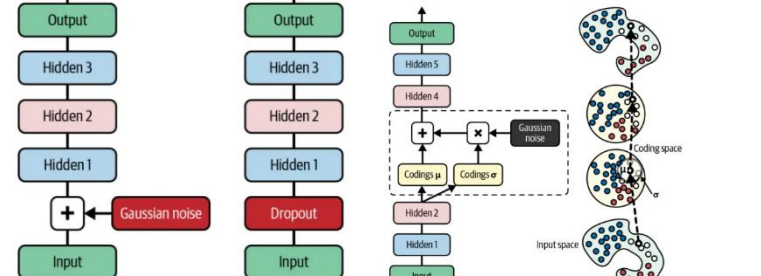


Figure 17-8. Denoising autoencoders, with Gaussian noise (left) or dropout (right)

Figure 17-11. A variational autoencoder (left) and an instance going through it (right)

## Topic 2 - Training DNNs

The vanishing gradients problem occurs when gradients become smaller and smaller as the algorithm progresses down to the lower layers. As a result, the Gradient Descent update leaves the lower layers' connection weights virtually unchanged, and training never converges to a good solution or the convergence is extremely slow. In other cases, we have the opposite problem, i.e., gradients become larger and larger, leading to the exploding gradients problem.

**Problem:**

- Combo of logistic sigmoid and weight initialisation Norm(0,1)
- Variance of output neurons was much greater than input → logistic mean is 0.5 and not 0
- Logistic func is saturated for large values → derivative becomes 0
- ReLU also "dies" as inputs may always become negative.

| Initialization | Activation functions                     | $\sigma^2$ (Normal) |
|----------------|------------------------------------------|---------------------|
| Glorot         | None, tanh, sigmoid, softmax             | $1/fan_{avg}$       |
| He             | ReLU, Leaky ReLU, ELU, GELU, Swish, Mish | $2/fan_{in}$        |
| LeCun          | SELU                                     | $1/fan_{in}$        |

**Batch Normalisation:** inserted before or after the activate function in each hidden layer to normalised the input to have mean=0 and variance=1. Must learn trainable parameters  $\gamma$  and  $\beta$  for  $z = \gamma \otimes \hat{x} + \beta$  where ( $\hat{x}$  is the  $z$  score of batch). After(keep bias) before(remove bias)

**Momentum:** uses **m** vector to keep track of gradients, acceleration rather than speed,  $\beta$  is how much of past to use and decays past.

**Nesterov Accelerated Gradient (momentum):** momentum but measures the gradient of loss function not at  $\theta$  but slightly ahead in the direction of the momentum at  $\theta + \beta \mathbf{m}$ , momentum vector points it in the right direction.

**AdaGrad:** scales the decent toward global minimum along steepest direction, assigns each parameter its own learning rate that adjusts automatically based on **s**, has effect of adaptive lr, DIS: performs well for quadratic problems but stops too early when training neural networks (lr scaled down so much).

**RMSProp:** slowing down problem from AdaGrad overcome, **s** is only from most recent gradient accumulations set by decay rate  $\rho$  (usually 0.9).

**Adam:** Adaptive moment estimation, combines momentum and RMSProp, estimates mean and variance of the error gradients to keep track of the exponentially decaying average of past squared gradients.

Uses same **m** and **s** calc but  $\beta_1$  is  $\beta$  (momentum) and  $\beta_2$  is  $\rho$  (RMS),  $\hat{m} = m/(1 - \beta_1)$ ,  $\hat{s} = s/(1 - \beta_2)$

**Adam Variants:** AdaMax, Nadam, AdamW (regularisation that reduces models weights at each training iteration)

| Class                            | Convergence speed | Convergence quality |
|----------------------------------|-------------------|---------------------|
| SGD                              | *                 | ***                 |
| SGD(momentum=...)                | **                | ***                 |
| SGD(momentum=..., nesterov=True) | **                | ***                 |
| AdaGrad                          | ***               | * (stops too early) |
| RMSprop                          | ***               | ** or ***           |
| Adam                             | ***               | ** or ***           |
| AdaMax                           | ***               | ** or ***           |
| Nadam                            | ***               | ** or ***           |
| AdamW                            | ***               | ** or ***           |

Table 11-2. Optimizer comparison.

**LR Scheduling:** balance between time and convergence.

**Piecewise Constant:** have steps

**Performance:** reduce on plateau

**1Cycle:** up then down

**L1 and L2 Regularization:** constrain NN connection weights at each training step.

**Dropout:** probability of input neuron being ignored in training.

**GAN:** Make two neural networks compete against each other to make them excel. Generator takes random distribution as input and outputs some data, kinda like decoder taking codings to make images look realistic to trick discriminator. Discriminator takes either a fake image from generator or real image as input and guesses if image is real or fake. Discriminator training: Trained on a mix of real and generated images using binary cross-entropy; only the discriminator's weights are updated. Generator training: Produces new fake images and aims to make the discriminator classify them as real; the discriminator's weights are frozen so only the generator learns.

## Topic 4 – Encoder Decoder

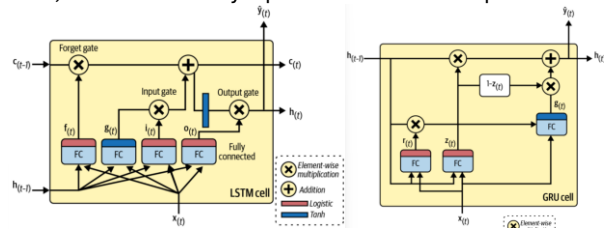
RNNs have symmetry with time, more recent time frame have more recent memory.

**1D Convolutional Layers:** feature map over sequence.

**RNN Limits:**

1. Struggle with long sequences which has vanishing/exploding gradient problem and they gradually forget first inputs.
2. Nonsaturating activations may not help as weights are same which could lead to exploding gradients

**LSTM:** Faster training convergence, detect long-term dependencies in data, remembers early inputs better **GRU:** Simplified LSTM cell.



## Topic 9 – Reinforcement Learning

Optimising rewards, an Agent makes observations and takes actions within an environment and in return receives rewards. Objective is to learn to act in a way that will maximise long term rewards. Policy is an algorithm used by agent to determine its actions. Methods:

**Brute force grid search:** try all combinations of policy params, policy search, if space large then needle in haystack.

**Genetic Algorithms:** survival of fittest, good policies make offspring **Policy Gradients with Optimisation technique:** can be hard coded rule based or neural network based (need to balance exploration/exploitation – example is multinomial sampling action based on output probability). NN policy caveats: if observations don't contain full state of environment, may need to consider past actions as well. If obs are noisy, use past few obs to estimate the most likely current state.

**Credit Assignment Problem:** Only guidance is through rewards which are typically sparse and delayed. Pole fell after 100 steps doesn't the last action is to blame. Hard to know which actions should be credited. Discounted rewards: discount rate  $\gamma$  at each step AFTER action.

Action Advantage: good actions may be followed by bad actions so the good action gets low score, average out actions good actions score better, normalise the scores (x-mean/std)

**Policy Gradient Alg REINFORCE:** optimise params of policy to follow gradients toward higher rewards (1) Let NN play game several episodes and at each step compute gradients that would make the chosen action more likely, but don't apply them yet. (2) then compute each actions score (action advantage). (3) multiply action advantage by gradient. (4) compute mean of resulting gradient vectors and use it to perform a gradient descent step.

**Markov Decision Processes:** stochastic process with fixed number of states with probabilities of transitioning dependent on chosen actions with some transitions giving rewards. (MC + actions + rewards).

**Bellman Optimality Equation (recursive)** is implemented with the *Value Iteration Algorithm (dynamic programming)*, estimates how good it is to be in state  $s$ .

**Q-Value Iteration (focus on state-action pairs):**  $Q^*(s, a)$  is the sum of discounted future rewards the agent can expect on average after it reaches the state  $s$  and chooses action  $a$ , but before it sees the outcome of this action, assuming it acts optimally after that action. After obtaining optimal Q-values, the optimal policy  $\pi^*(s) = \max_a Q^*(s, a)$

**Temporal Difference Learning:** transition probs and rewards are often not known, agent must experience each state and transition. where  $\alpha$  is the learning rate;  $r + V_k(s')$  is called the TD target;  $\delta_k(s, r, s')$  is called the TD error. Keeps running avg of immediate rewards + later expected rewards. Similar to SGD (one sample at time, need to reduce  $\alpha$ ).

**Q-Learning:** Doesn't know T or R, updates Q using observed transitions, **Off-policy:** policy being trained is not the one used during training. DIS: doesn't scale well to medium+ MDPs as too many params

**$\epsilon$ -Greedy Policy:** balance exploit/explore,  $\epsilon$  decays overtime, with prob  $\epsilon$  being random action (explore) and prob  $1-\epsilon$  being best action (exploit) **Approximate & Deep Q-Learning:** no feature engineering and can learn rules. Idea: Use manageable number of params. Deep Q-network is DNN to estimate Q-values, DQN for approx. Q-learning is Deep Q-Learning. Target value is the **best expected total reward** achievable from the current state and action - combining the immediate reward ( $r$ ) and the best discounted estimate of future rewards ( $\gamma \times \max Q(s', a)$ )).

**Catastrophic forgetting:** forget prev learned info during training.

## Topic 5 – Autoencoders

Learning efficient representations of the input data called latent representation codings without supervision

Usually have much lower dimensionality that the data useful for dimensionality reduction

Act as feature detectors

GENERATIVE models which are capable of randomly generating new data

Undercomplete autoencoder is when internal representation is lower dimension, overcomplete is when its larger.

Auto encoders are NOT linear due to the activation functions

Stacked autoencoders are deep autoencoders

**PCA:** Use no activation functions

**T-SNE:** High dimensional data to low dimensional manifold.

**Unsupervised pretraining using stacked AEs:** use the encoder with the output layer on top.

**Tying weights:** Use same weights for encoder and decoder but you transpose them (halves number of weights) -> limits risk of overfitting and speeds up training.

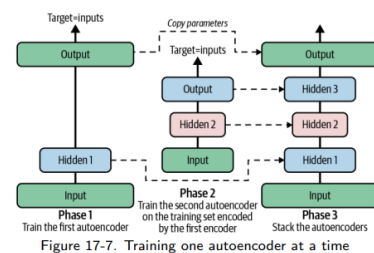


Figure 17-7. Training one autoencoder at a time

- **Phase 1:** Train the first autoencoder to learn a compressed representation  $X_{(1)}$  of the inputs.
- **Phase 2:** Using  $X_{(1)}$  as input data, train the second autoencoder to learn an even more compressed representation  $X_{(2)}$ .
- **Phase 3:** Finally, we build a big sandwich using all these autoencoders.

## Topic 6&7 – Convolutional AE, Denoising AE, Variational AEs, GANs

**Convolutional AEs:** best for auto encoding images

- Encoder is regular CNN
- Decoder is reverse, upscale image through transpose convolutional layers to original dimensions

**Denoising AEs:** another way to force AE to learn useful features is to add noise to its inputs and training it to recover the original, noise free inputs. Noise can be gaussian noise or dropout.

**Variational Autoencoders:** probabilistic and generative

- Random sample from a learned distribution which can then be used to reconstructed a new sample
- Encoder: outputs two vectors Mean  $\mu$  and log variance  $\log(\sigma^2)$
- Reparameterization trick: sample from  $z = \mu + \sigma \odot \epsilon$  where  $\epsilon \sim N(0, I)$ , allows gradient based training through stochastic sampling
- Decoder: takes  $z$  and reconstructs  $\hat{x}$
- Loss = Kullback-Leibler divergence + reconstruction loss

- During training, the cost function (see later) pushes the codings to gradually migrate within the coding space (also called the *latent space*) to end up looking like a cloud of Gaussian points.

- Drawing samples from this latent space thus allows the decoder to reconstruct the inputs (using the two hidden layers *Hidden 3* and *Hidden 4*).

- The *variational autoencoder's latent loss* is defined as:

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2],$$

where  $n$  is the codings' dimension;  $\mu_i$  and  $\sigma_i$  are, respectively, the  $i^{\text{th}}$  components of  $\mu$  and  $\sigma$ .

- A common tweak adopted in practice is to make the encoder output  $\gamma$  (defined as  $\log(\sigma^2)$ ) rather than  $\sigma$ , so that the algorithm is more numerically stable and the training is faster. So, the *variational autoencoder's latent loss* becomes

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \gamma_i - \exp(\gamma_i) - \mu_i^2].$$

**GAN Difficulties:** challenging due to unstable dynamics between generator and discriminator. (1) Nash equilibrium: ideally both NNs should converge where neither can improve without the other changing, in practice this is rare. (2) Mode collapse: NNs produce limited varieties of outputs, losing generalisation. (3) Instability: both networks continuously adapt to outsmart each other causing oscillating parameters.